

ex13_solution

02477 Bayesian Machine Learning - Exercise 13¶

Overview¶

In this exercise we will study Bayesian-inspired methods for deep learning. The exercise is divided into the following parts:

Content

- Part 1: Introduction and theory
- Part 2: Fitting a neural network to a toy problem with MAP-inference
- Part 3: Last layer Laplace approximations (LLA)
- Part 4: Deep ensembles
- Part 5: Modelling the Concrete dataset from the UCI repository

Note: The exercise contains several **discussion questions**, which are questions, where are supposed to actively experiment with the code and/or reason with the equations to arrive at the relevant conclusions. This also means that we won't provide a specific solution for this task. However, you are more than welcome to check your understanding and your conclusions with the TAs. Instead of proving the full description for every discussion question, we simply tag it with: **[Discussion question]** after the question.

Note: If you find yourself spending more than 30minutes on a single task, consider asking for help or look up at the solution for inspiration and move forward.

Note: This notebook will also make use of the package `optax` for optimization.

Part 1: Introduction and theory¶

In this exercise, we will explore Bayesian-inspired methods for deep learning. The core concept in Bayesian methods is to compute weighted averages wrt. the posterior distribution of the parameters instead of "betting everything" on a single set of parameters, i.e. **a point estimate**. We will focus on supervised learning with probabilistic models of the form

$$\begin{aligned} p(\mathbf{y}, \mathbf{w}) &= \prod_{n=1}^N p(y_n | \mathbf{w}) p(\mathbf{w}) \\ \tag{1} \end{aligned}$$

where $p(y_n | \mathbf{w})$ is the likelihood for the n -th observation, which is parametrized by a neural network with parameters $\mathbf{w}$. Ideally, we would like to compute the posterior distribution $p(\mathbf{w} | \mathbf{y})$ and use this to compute the posterior predictive distribution, but this is generally a very difficult task for neural network models due to the high-dimensional nature of the models, the complexity posterior geometry, and scale of modern datasets.

Non-linear regression models with heteroscedastic noise

In this exercise, we will study non-linear regression models with Gaussian likelihoods of the form

$$\begin{aligned} p(y_n | \mathbf{w}, \mathbf{x}_n) &= \mathcal{N}(y_n | \mu(\mathbf{w})(\mathbf{x}_n), \sigma^2(\mathbf{w})(\mathbf{x}_n)). \\ \tag{2} \end{aligned}$$

In this model, we assume **heteroscedasticity**, i.e. we allow the noise variance of the observations to depend on the input $\mathbf{x}$. This should be contrasted with **homoscedastic models**, where the noise variance is assumed to be constant, i.e. $\sigma^2(\mathbf{x}_n) = \sigma^2_0$. Note that this is the **variance of the likelihood**, i.e. the aleatoric component of the uncertainty, so this should not be confused with epistemic uncertainty, which can also depend on $\mathbf{x}$.

We will parameterize this likelihood using a neural network with two outputs, where the first output represents the mean function of the Gaussian distribution and the second represent the log variance of the Gaussian distribution. That is, if $f(\mathbf{x} | \mathbf{w}): \mathbb{R}^D \rightarrow \mathbb{R}^2$ is a neural network with parameters $\mathbf{w}$ and two output nodes, then

$$\begin{aligned} \mu(\mathbf{w})(\mathbf{x}) &= f_1(\mathbf{x} | \mathbf{w}) \\ \log \sigma^2(\mathbf{w})(\mathbf{x}) &= f_2(\mathbf{x} | \mathbf{w}). \end{aligned}$$

This construction implies that $\sigma^2(\mathbf{w})(\mathbf{x}) = e^{f_2(\mathbf{x} | \mathbf{w})} > 0$, which ensures that the predicted noise variance will always be strictly positive no matter what the neural network outputs.

We will use a fully-connected neural network with two hidden layers:

$$\begin{aligned} \mathbf{z}_1 &= \mathbf{h}(\mathbf{W}_0 \mathbf{x} + \mathbf{b}_0), \\ \mathbf{z}_2 &= \mathbf{h}(\mathbf{W}_1 \mathbf{z}_1 + \mathbf{b}_1), \\ \mathbf{f} &= \mathbf{W}_2 \mathbf{z}_2 + \mathbf{b}_2, \end{aligned}$$

where $\mathbf{f} \in \mathbb{R}^2$ and $\mathbf{W}_0$ and $\mathbf{b}_0$ are the weights and biases for the first layer etc and $\mathbf{h}$ is a non-linear activation function, which we will take to be the tangent

hyperbolic function. We assume an isotropic Gaussian prior on all parameters, i.e. if $\mathbf{w} = \left\{ \mathbf{W}_0, \mathbf{b}_0, \mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2 \right\}$ denotes all the parameters of the network, then

$$\begin{aligned} p(\mathbf{w}) = \mathcal{N}(\mathbf{w} | \mathbf{0}, \alpha^{-1} \mathbf{I}) \end{aligned} \quad \text{for } \alpha > 0.$$

Approximate inference

In Bayesian deep learning, we aim to find a tractable approximation $q(\mathbf{w})$ such that we can approximate the posterior predictive distribution for a new point $\mathbf{y}^*$ as follows

$$\begin{aligned} p(\mathbf{y}^* | \mathbf{y}, \mathbf{x}^*) \approx \int p(\mathbf{y}^* | \mathbf{w}, \mathbf{x}^*) q(\mathbf{w}) d\mathbf{w} \end{aligned} \quad \text{tag{6}}$$

For example, for Laplace approximations and Gaussian variational approximations, we have $q(\mathbf{w}) = \mathcal{N}(\mathbf{w} | \mathbf{m}, \mathbf{V})$ and the integral in eq. (6) can be approximated using Monte Carlo sampling

$$\begin{aligned} p(\mathbf{y}^* | \mathbf{y}, \mathbf{x}^*) \approx \frac{1}{S} \sum_{i=1}^S p(\mathbf{y}^* | \mathbf{w}^{(i)}, \mathbf{x}^*), \quad \text{where } \mathbf{w}^{(i)} \sim q(\mathbf{w}) \end{aligned} \quad \text{tag{7}}$$

Deep Ensembles

An alternative to Gaussian posterior approximations is **Deep Ensembles** (DE), where we simply fit our model S -times from different initial parameters (e.g. by using different seeds) and treat the resulting parameters vectors $\mathbf{w}^{(i)}$ for $i = 1, \dots, S$ as pseudo-Monte Carlo samples corresponding to a posterior approximation of the form $q_{\text{DE}}(\mathbf{w}) = \frac{1}{S} \sum_{i=1}^S \delta(\mathbf{w} - \mathbf{w}^{(i)})$, where $\delta(\cdot)$ is Dirac's delta function. Thus, for deep ensembles, we approximate the posterior predictive distribution as follows

$$\begin{aligned} p(\mathbf{y}^* | \mathbf{y}, \mathbf{x}^*) \approx \int p(\mathbf{y}^* | \mathbf{w}, \mathbf{x}^*) q_{\text{DE}}(\mathbf{w}) d\mathbf{w} = \frac{1}{S} \sum_{i=1}^S p(\mathbf{y}^* | \mathbf{w}^{(i)}, \mathbf{x}^*). \end{aligned} \quad \text{tag{8}}$$

MAP-inference is equivalent to a "degenerate" deep ensemble with $S = 1$ and therefore, we can think of MAP-inference as approximate posterior inference with the degenerate approximation $q_{\text{MAP}}(\mathbf{w}) = \delta(\mathbf{w} - \mathbf{w}_{\text{MAP}})$:

$$\begin{aligned} p(\mathbf{y}^* | \mathbf{y}, \mathbf{x}^*) \approx \int p(\mathbf{y}^* | \mathbf{w}, \mathbf{x}^*) q_{\text{MAP}}(\mathbf{w}) d\mathbf{w} = p(\mathbf{y}^* | \mathbf{w}_{\text{MAP}}, \mathbf{x}^*), \end{aligned} \quad \text{tag{8b}}$$

which we also know as a **plugin approximation**.

Task 1.1: Suppose you have trained a deep ensemble with $S = 10$ models and let $\mathbf{f}^{(i)} \in \mathbb{R}^2$ denote the output of the neural network for the i 'th model for a given input $\mathbf{x}^*$. Determine the expression for the resulting approximate posterior predictive distribution.

Solution

Using eq. (8) we have that
$$p(\mathbf{y}^* | \mathbf{y}, \mathbf{x}^*) \approx \frac{1}{S} \sum_{i=1}^S p(\mathbf{y}^* | \mathbf{w}^{(i)}, \mathbf{x}^*),$$
 and inserting the distribution from eq. (2) yields
$$p(\mathbf{y}^* | \mathbf{y}, \mathbf{x}^*) \approx \frac{1}{S} \sum_{i=1}^S \mathcal{N}(\mathbf{y}^* | \mathbf{f}_1^{(i)}, \mathbf{e}^{(\mathbf{f}_2^{(i)})}).$$

Hence, the resulting posterior predictive distribution is a Gaussian mixture model with means $\{\mathbf{f}_1^{(i)}\}_{i=1}^S$, variances $\{\mathbf{e}^{(\mathbf{f}_2^{(i)})}\}_{i=1}^S$ and uniform mixing weights $\frac{1}{S}$.

End of solution

Last-layer Laplace approximations

In this exercise, we will compare standard **MAP-inference** with **deep ensembles** and **Last Layer Laplace Approximations** (LLLA). To construct a Laplace approximation, we need to evaluate the Hessian of the log joint distribution, which can be prohibitively expensive for even moderately sized networks. The LLLA approximation is a cheap alternative, where we treat all layers as deterministic except for the very last layer. That is, for our network with 2 hidden layers, we treat the parameters $\mathbf{W}_0, \mathbf{b}_0, \mathbf{W}_1$ and $\mathbf{b}_1$ as deterministic and only apply the Laplace approximation for the parameters of the last layer, i.e. $\mathbf{W}_2$ and $\mathbf{b}_2$. Consequently, we only need to estimate a much smaller Hessian matrix. Furthermore, LLLA approximation has the additional benefits that it can easily be added *post-hoc* to a pretrained model.

Let L be the number of layers and let $\mathbf{z}_2 \in \mathbb{R}^H$, then we can separate the network into two functions: A function $f_{1:L-1}$ that maps from the input to output of the second hidden layer, i.e. $\mathbf{z}_2 = f_{1:L-1}(\mathbf{x}) : \mathbb{R}^D \rightarrow \mathbb{R}^H$, and a function f_L that maps $\mathbf{z}_2$ to the output of the network, i.e. $f_L : \mathbb{R}^H \rightarrow \mathbb{R}^O$ defined by

$$\begin{aligned} f_L(\mathbf{z}_2 | \mathbf{w}_L) &= \mathbf{W}_2 \mathbf{z}_2 + \mathbf{b}_2, \end{aligned}$$

where $\mathbf{w}_L = \{\mathbf{W}_2, \mathbf{b}_2\}$ is the parameters of the layer and $\mathbf{w}_{1:L-1}$ is the parameters of the rest of the network. That is, to apply LLLA, we first compute the MAP estimate of the full network, i.e. $\mathbf{w}_{1:L-1}^{\text{MAP}}$,

but we only have to compute the Hessian wrt. $\mathbf{w}_L$.

Evaluation

A common way to evaluate probabilistic models is using the average *log predictive density* (LPD) for a dataset (arguments for this metric can be found in Evaluating Predictive Uncertainty Challenge, Quiñero-Candela et al., 2006). For a set of posteriors samples $\{\mathbf{w}^{(i)}\}_{i=1}^S$ (no matter whether they come from a Laplace approximation or deep ensembles etc), we can compute the posterior distribution of y by computing the output of the network for each of the weight samples $\mathbf{w}^{(i)}$ to get

$$\left\{ f^{(i)}(\mathbf{x}^*) \right\}_{i=1}^S$$

for a test point $\mathbf{x}^*$. Finally, we can use these samples to measure the log predictive density as follows

$$\text{LPD}(y^*) = \log p(y^* | \mathbf{y}, \mathbf{x}^*) \approx \log \left[\frac{1}{S} \sum_{i=1}^S p(y^* | \mathbf{w}^{(i)}, \mathbf{x}^*) \right]$$

For our specific model, which outputs the mean and log variance of the Gaussian predictive distribution, the log predictive density for y^* becomes

$$\text{LPD}(y^*) = \log \left[\frac{1}{S} \sum_{i=1}^S \mathcal{N}(y^* | f_1^{(i)}(\mathbf{x}^*), e^{f_2^{(i)}(\mathbf{x}^*)}) \right]$$

Hence, for a test of M observations, the averaged LPD becomes

$$\text{LPD} = \frac{1}{M} \sum_{m=1}^M \log \left[\frac{1}{S} \sum_{i=1}^S \mathcal{N}(y_m^* | f_1^{(i)}(\mathbf{x}_m^*), e^{f_2^{(i)}(\mathbf{x}_m^*)}) \right]$$

Note: To avoid numeric underflow, the logarithm of the inner sum is often evaluated using the `logsumexp` 'trick'.

Part 2: Fitting a neural network to a toy problem with MAP-inference¶

Let's load a small toy dataset:

```
<matplotlib.legend.Legend at 0x7f6e481cdd90>
```



Task 2.1: Consider the dataset above. Why would the assumption of **homoscedastic noise** be inappropriate here? **[Discussion question]**

Task 2.2: Complete the implementation of the log likelihood function below.

The following cell will compute the MAP-solution of the neural network by optimizing the log joint distribution wrt. $\mathbf{w}$:

```

Itt = 100/2000 (5.0%), log joint = -634.79
Itt = 200/2000 (10.0%), log joint = -612.35
Itt = 300/2000 (15.0%), log joint = -666.15
Itt = 400/2000 (20.0%), log joint = -563.53
Itt = 500/2000 (25.0%), log joint = -554.85
Itt = 600/2000 (30.0%), log joint = -555.36
Itt = 700/2000 (35.0%), log joint = -555.61
Itt = 800/2000 (40.0%), log joint = -557.70
Itt = 900/2000 (45.0%), log joint = -551.67
Itt = 1000/2000 (50.0%), log joint = -552.44
Itt = 1100/2000 (55.0%), log joint = -552.65
Itt = 1200/2000 (60.0%), log joint = -558.69
Itt = 1300/2000 (65.0%), log joint = -554.79
Itt = 1400/2000 (70.0%), log joint = -551.29
Itt = 1500/2000 (75.0%), log joint = -549.78
Itt = 1600/2000 (80.0%), log joint = -549.74
Itt = 1700/2000 (85.0%), log joint = -550.10
Itt = 1800/2000 (90.0%), log joint = -549.16
Itt = 1900/2000 (95.0%), log joint = -552.22
Itt = 2000/2000 (100.0%), log joint = -548.39

```

We can now use the function `model_MAP.predict` to compute a forward pass through the network. That is, it evaluates $f(\mathbf{x})$ for the input points in the array `Xp` using $\mathbf{w}_{\text{MAP}}$:

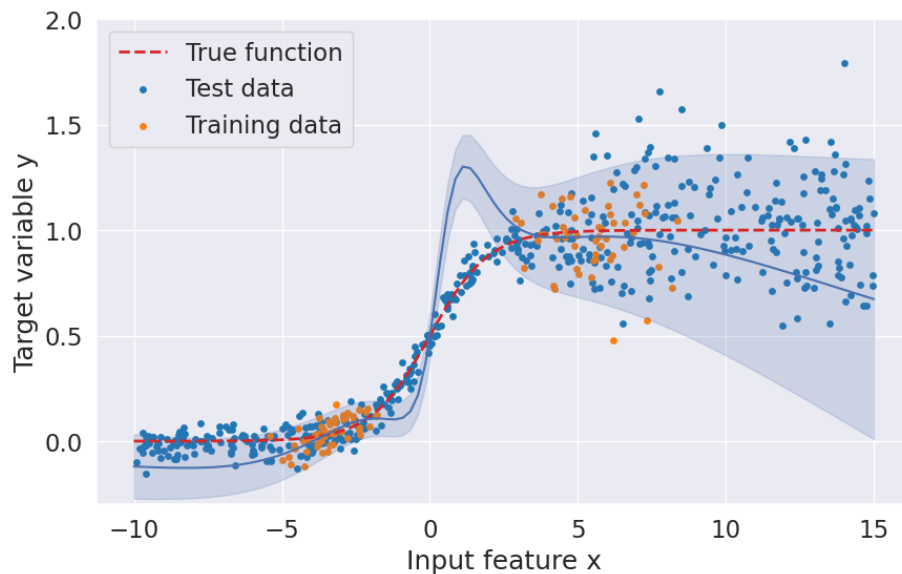
```
shape of Xp: (100, 1)
shape of pred: (100, 2)
```

The output variable `pred` is an $N \times 2$ array because the network has two outputs.

For the MAP-method, we can easily compute the predictive mean and resulting intervals:

- For each input point $\mathbf{x}$, we predict a mean value $\mu(\mathbf{x})$ and a variance $\sigma^2(\mathbf{x})$ of a Gaussian.
- A 95%-interval for a Gaussian $\mathcal{N}(x|m, v)$ is approximately $[m - 1.96\sqrt{v}, m + 1.96\sqrt{v}]$.

<matplotlib.legend.Legend at 0x7f6d883ddac0>



Task 2.3: How can you tell from the resulting fit that we have used a heteroscedastic model? Comment on the quality of the fit. Is it reasonable? [Discussion question]

Next, let's compute the average LPD for both the training and test set.

Task 2.4: Compute the average LPD for the training and the test sets.

Hints:

- Recall that $q_{\{\text{MAP}\}}(\mathbf{w}) = \delta(\mathbf{w} - \mathbf{w}_{\{\text{MAP}\}})$ and use eq. (8b) and (11).

Solution

Train LPD MAP: 1.03

Test LPD MAP: -1.22

End of solution

Part 3: Last layer Laplace approximations (LLA)¶

Next, we will study the last-layer Laplace approximation. The model class `NeuralNetwork` has two convenient functions for this purpose: `first_layers(params, X)` and `last_layer(param_last_layer, z)`, which implements $f_{\{1:L-1\}}$ and f_L , respectively.

The cell below provides a simple implementation of LLA. A couple of details of about the implementation:

- The `model.params` variable stores all parameters of the network as a nested list: $[[\mathbf{W}_0, \mathbf{b}_0], [\mathbf{W}_1, \mathbf{b}_1], [\mathbf{W}_2, \mathbf{b}_2]]$.
- The function `ravel_pytree` takes a nested list of parameters as input and returns 1) a flattened version of the list (i.e. a vector), and 2) an `unflatten` function to the reverse this operation.
- We will use the flattened parameter vector for computing the Hessian using `JaX`.

Thus, the array `w_MAP_flat` and the matrix `S_LLA` contain the mean and covariance matrix of the posterior approximation of the last-layer, respectively.

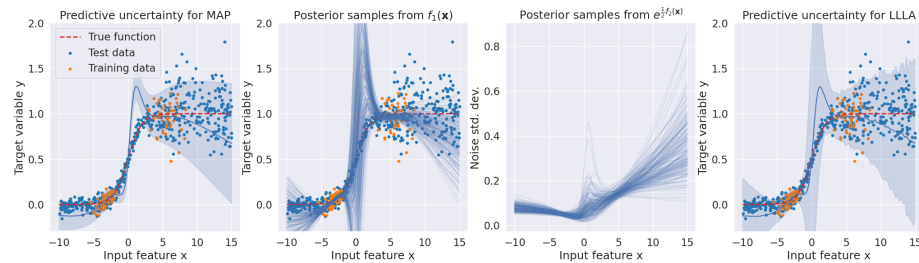
Task 3.1: Fill in the missing pieces in the implementation below:

shape of `mu_LLA_pred`: (1000, 100)

shape of `v_LLA_pred`: (1000, 100)

The two arrays above now contains $S = 1000$ samples of the mean and variance of the Gaussian distribution for each of the 100 points in the array `Xp`. We can use those for plotting the predictive distribution.

Let's compare the predictions for MAP and LLA:



Task 3.2: Comment on the resulting fits. How does the MAP and LLLA compare? How well do the models fit in region with training data and in regions without training data? Does the uncertainty look reasonable? [**Discussion question**]

Task 3.3: Which of the four plots visualizes (some degree of) aleatoric uncertainty, epistemic uncertainty or a combination? [**Discussion question**]

Let's now compute the training and test LPD for LLLA.

Task 3.4: Compute the average LPD for the training and test set for the LLLA approximation. How does the LLLA approximation compare with the MAP solution in terms of LPD?

Solution

LPD train LLLA: 1.03

LPD test LLLA: 0.28

End of solution

Part 4: Deep ensembles ¶

Finally, we will implement a deep ensemble with 10 models and compare to MAP and LLLA solutions.

Below you are given a simple template for implementing a deep ensemble:

Training model 1/10

```
Itt = 100/2000 (5.0%), log joint = -634.79
Itt = 200/2000 (10.0%), log joint = -612.35
Itt = 300/2000 (15.0%), log joint = -666.15
Itt = 400/2000 (20.0%), log joint = -563.53
Itt = 500/2000 (25.0%), log joint = -554.85
Itt = 600/2000 (30.0%), log joint = -555.36
Itt = 700/2000 (35.0%), log joint = -555.61
Itt = 800/2000 (40.0%), log joint = -557.70
Itt = 900/2000 (45.0%), log joint = -551.67
Itt = 1000/2000 (50.0%), log joint = -552.44
Itt = 1100/2000 (55.0%), log joint = -552.65
Itt = 1200/2000 (60.0%), log joint = -558.69
Itt = 1300/2000 (65.0%), log joint = -554.79
Itt = 1400/2000 (70.0%), log joint = -551.29
Itt = 1500/2000 (75.0%), log joint = -549.78
Itt = 1600/2000 (80.0%), log joint = -549.74
Itt = 1700/2000 (85.0%), log joint = -550.10
Itt = 1800/2000 (90.0%), log joint = -549.16
Itt = 1900/2000 (95.0%), log joint = -552.22
Itt = 2000/2000 (100.0%), log joint = -548.39
```

Training model 2/10

```

Itt = 100/2000 (5.0%), log joint = -828.83
Itt = 200/2000 (10.0%), log joint = -700.86
Itt = 300/2000 (15.0%), log joint = -580.69
Itt = 400/2000 (20.0%), log joint = -564.88
Itt = 500/2000 (25.0%), log joint = -561.88
Itt = 600/2000 (30.0%), log joint = -557.63
Itt = 700/2000 (35.0%), log joint = -558.94
Itt = 800/2000 (40.0%), log joint = -556.24
Itt = 900/2000 (45.0%), log joint = -557.20
Itt = 1000/2000 (50.0%), log joint = -555.79
Itt = 1100/2000 (55.0%), log joint = -566.58
Itt = 1200/2000 (60.0%), log joint = -555.34
Itt = 1300/2000 (65.0%), log joint = -567.18
Itt = 1400/2000 (70.0%), log joint = -553.82
Itt = 1500/2000 (75.0%), log joint = -553.29
Itt = 1600/2000 (80.0%), log joint = -552.86
Itt = 1700/2000 (85.0%), log joint = -553.53
Itt = 1800/2000 (90.0%), log joint = -552.23
Itt = 1900/2000 (95.0%), log joint = -551.98
Itt = 2000/2000 (100.0%), log joint = -551.79
Training model 3/10
Itt = 100/2000 (5.0%), log joint = -794.93
Itt = 200/2000 (10.0%), log joint = -779.83
Itt = 300/2000 (15.0%), log joint = -765.07
Itt = 400/2000 (20.0%), log joint = -748.15
Itt = 500/2000 (25.0%), log joint = -717.21
Itt = 600/2000 (30.0%), log joint = -677.71
Itt = 700/2000 (35.0%), log joint = -647.99
Itt = 800/2000 (40.0%), log joint = -620.07
Itt = 900/2000 (45.0%), log joint = -591.62
Itt = 1000/2000 (50.0%), log joint = -570.36
Itt = 1100/2000 (55.0%), log joint = -561.44
Itt = 1200/2000 (60.0%), log joint = -559.30
Itt = 1300/2000 (65.0%), log joint = -558.24
Itt = 1400/2000 (70.0%), log joint = -557.37
Itt = 1500/2000 (75.0%), log joint = -556.82
Itt = 1600/2000 (80.0%), log joint = -556.36
Itt = 1700/2000 (85.0%), log joint = -559.08
Itt = 1800/2000 (90.0%), log joint = -555.63
Itt = 1900/2000 (95.0%), log joint = -555.31
Itt = 2000/2000 (100.0%), log joint = -555.47
Training model 4/10
Itt = 100/2000 (5.0%), log joint = -557.17
Itt = 200/2000 (10.0%), log joint = -551.61
Itt = 300/2000 (15.0%), log joint = -551.93
Itt = 400/2000 (20.0%), log joint = -550.19

```

```

Itt = 500/2000 (25.0%), log joint = -549.84
Itt = 600/2000 (30.0%), log joint = -547.82
Itt = 700/2000 (35.0%), log joint = -546.84
Itt = 800/2000 (40.0%), log joint = -546.40
Itt = 900/2000 (45.0%), log joint = -549.65
Itt = 1000/2000 (50.0%), log joint = -555.13
Itt = 1100/2000 (55.0%), log joint = -544.72
Itt = 1200/2000 (60.0%), log joint = -554.72
Itt = 1300/2000 (65.0%), log joint = -543.47
Itt = 1400/2000 (70.0%), log joint = -543.68
Itt = 1500/2000 (75.0%), log joint = -542.56
Itt = 1600/2000 (80.0%), log joint = -543.28
Itt = 1700/2000 (85.0%), log joint = -541.78
Itt = 1800/2000 (90.0%), log joint = -541.93
Itt = 1900/2000 (95.0%), log joint = -541.16
Itt = 2000/2000 (100.0%), log joint = -541.23
Training model 5/10
Itt = 100/2000 (5.0%), log joint = -639.11
Itt = 200/2000 (10.0%), log joint = -560.86
Itt = 300/2000 (15.0%), log joint = -550.12
Itt = 400/2000 (20.0%), log joint = -549.36
Itt = 500/2000 (25.0%), log joint = -572.99
Itt = 600/2000 (30.0%), log joint = -548.90
Itt = 700/2000 (35.0%), log joint = -552.51
Itt = 800/2000 (40.0%), log joint = -549.37
Itt = 900/2000 (45.0%), log joint = -546.73
Itt = 1000/2000 (50.0%), log joint = -549.09
Itt = 1100/2000 (55.0%), log joint = -545.98
Itt = 1200/2000 (60.0%), log joint = -548.52
Itt = 1300/2000 (65.0%), log joint = -545.18
Itt = 1400/2000 (70.0%), log joint = -548.66
Itt = 1500/2000 (75.0%), log joint = -544.63
Itt = 1600/2000 (80.0%), log joint = -562.58
Itt = 1700/2000 (85.0%), log joint = -544.31
Itt = 1800/2000 (90.0%), log joint = -543.84
Itt = 1900/2000 (95.0%), log joint = -543.68
Itt = 2000/2000 (100.0%), log joint = -545.15
Training model 6/10
Itt = 100/2000 (5.0%), log joint = -755.72
Itt = 200/2000 (10.0%), log joint = -607.94
Itt = 300/2000 (15.0%), log joint = -578.71
Itt = 400/2000 (20.0%), log joint = -564.26
Itt = 500/2000 (25.0%), log joint = -559.32
Itt = 600/2000 (30.0%), log joint = -558.54
Itt = 700/2000 (35.0%), log joint = -562.13
Itt = 800/2000 (40.0%), log joint = -598.76

```

```

Itt = 900/2000 (45.0%), log joint = -556.47
Itt = 1000/2000 (50.0%), log joint = -560.75
Itt = 1100/2000 (55.0%), log joint = -555.10
Itt = 1200/2000 (60.0%), log joint = -555.50
Itt = 1300/2000 (65.0%), log joint = -555.69
Itt = 1400/2000 (70.0%), log joint = -555.44
Itt = 1500/2000 (75.0%), log joint = -553.05
Itt = 1600/2000 (80.0%), log joint = -553.13
Itt = 1700/2000 (85.0%), log joint = -553.85
Itt = 1800/2000 (90.0%), log joint = -552.93
Itt = 1900/2000 (95.0%), log joint = -551.74
Itt = 2000/2000 (100.0%), log joint = -551.34
Training model 7/10
Itt = 100/2000 (5.0%), log joint = -699.73
Itt = 200/2000 (10.0%), log joint = -640.22
Itt = 300/2000 (15.0%), log joint = -557.40
Itt = 400/2000 (20.0%), log joint = -558.18
Itt = 500/2000 (25.0%), log joint = -556.52
Itt = 600/2000 (30.0%), log joint = -557.04
Itt = 700/2000 (35.0%), log joint = -552.91
Itt = 800/2000 (40.0%), log joint = -553.73
Itt = 900/2000 (45.0%), log joint = -552.04
Itt = 1000/2000 (50.0%), log joint = -552.70
Itt = 1100/2000 (55.0%), log joint = -551.21
Itt = 1200/2000 (60.0%), log joint = -550.71
Itt = 1300/2000 (65.0%), log joint = -550.99
Itt = 1400/2000 (70.0%), log joint = -561.35
Itt = 1500/2000 (75.0%), log joint = -549.33
Itt = 1600/2000 (80.0%), log joint = -549.59
Itt = 1700/2000 (85.0%), log joint = -548.67
Itt = 1800/2000 (90.0%), log joint = -548.46
Itt = 1900/2000 (95.0%), log joint = -547.96
Itt = 2000/2000 (100.0%), log joint = -547.79
Training model 8/10
Itt = 100/2000 (5.0%), log joint = -785.25
Itt = 200/2000 (10.0%), log joint = -758.44
Itt = 300/2000 (15.0%), log joint = -714.61
Itt = 400/2000 (20.0%), log joint = -650.74
Itt = 500/2000 (25.0%), log joint = -580.24
Itt = 600/2000 (30.0%), log joint = -1022.88
Itt = 700/2000 (35.0%), log joint = -554.69
Itt = 800/2000 (40.0%), log joint = -552.70
Itt = 900/2000 (45.0%), log joint = -552.43
Itt = 1000/2000 (50.0%), log joint = -558.02
Itt = 1100/2000 (55.0%), log joint = -554.99
Itt = 1200/2000 (60.0%), log joint = -552.11

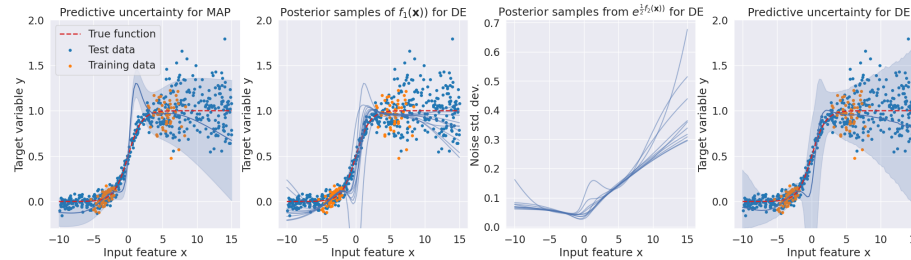
```

```

Itt = 1300/2000 (65.0%), log joint = -551.62
Itt = 1400/2000 (70.0%), log joint = -554.21
Itt = 1500/2000 (75.0%), log joint = -554.95
Itt = 1600/2000 (80.0%), log joint = -550.90
Itt = 1700/2000 (85.0%), log joint = -553.55
Itt = 1800/2000 (90.0%), log joint = -550.69
Itt = 1900/2000 (95.0%), log joint = -562.36
Itt = 2000/2000 (100.0%), log joint = -550.57
Training model 9/10
Itt = 100/2000 (5.0%), log joint = -732.86
Itt = 200/2000 (10.0%), log joint = -613.29
Itt = 300/2000 (15.0%), log joint = -596.98
Itt = 400/2000 (20.0%), log joint = -562.26
Itt = 500/2000 (25.0%), log joint = -555.61
Itt = 600/2000 (30.0%), log joint = -557.84
Itt = 700/2000 (35.0%), log joint = -554.07
Itt = 800/2000 (40.0%), log joint = -553.35
Itt = 900/2000 (45.0%), log joint = -552.63
Itt = 1000/2000 (50.0%), log joint = -554.15
Itt = 1100/2000 (55.0%), log joint = -557.22
Itt = 1200/2000 (60.0%), log joint = -553.55
Itt = 1300/2000 (65.0%), log joint = -553.52
Itt = 1400/2000 (70.0%), log joint = -550.47
Itt = 1500/2000 (75.0%), log joint = -550.77
Itt = 1600/2000 (80.0%), log joint = -558.41
Itt = 1700/2000 (85.0%), log joint = -550.43
Itt = 1800/2000 (90.0%), log joint = -550.37
Itt = 1900/2000 (95.0%), log joint = -551.43
Itt = 2000/2000 (100.0%), log joint = -547.90
Training model 10/10
Itt = 100/2000 (5.0%), log joint = -602.36
Itt = 200/2000 (10.0%), log joint = -947.99
Itt = 300/2000 (15.0%), log joint = -557.08
Itt = 400/2000 (20.0%), log joint = -553.57
Itt = 500/2000 (25.0%), log joint = -553.44
Itt = 600/2000 (30.0%), log joint = -552.63
Itt = 700/2000 (35.0%), log joint = -552.37
Itt = 800/2000 (40.0%), log joint = -552.78
Itt = 900/2000 (45.0%), log joint = -595.31
Itt = 1000/2000 (50.0%), log joint = -552.03
Itt = 1100/2000 (55.0%), log joint = -572.62
Itt = 1200/2000 (60.0%), log joint = -554.26
Itt = 1300/2000 (65.0%), log joint = -551.60
Itt = 1400/2000 (70.0%), log joint = -548.68
Itt = 1500/2000 (75.0%), log joint = -549.32
Itt = 1600/2000 (80.0%), log joint = -548.18

```

Itt = 1700/2000 (85.0%), log joint = -547.89
 Itt = 1800/2000 (90.0%), log joint = -546.82
 Itt = 1900/2000 (95.0%), log joint = -551.74
 Itt = 2000/2000 (100.0%), log joint = -545.86



Task 4.1: How does the DE approximation compare qualitatively to the MAP solution? In regions with training data? In regions without training data? [Discussion question]

Task 4.2: Compute the average log predictive density for the training and test set for the deep ensemble.

Hints:

- Use equation (8) and (11)

Solution

LPD train DE: 1.04

LPD test DE: 0.14

End of solution

Task 4.3: How does the three methods compare in terms of LPD for the toy data? [Discussion question]

Part 5: Modelling the Concrete dataset from the UCI repository ¶

In this part, we will test this model with three types of inference: MAP, LLLA and DE on a real dataset. We will work with the Concrete dataset from the UCI repository.

The goal is to predict the strength of concrete based on a set of input features. See the link below for more information:

<https://archive.ics.uci.edu/ml/datasets/concrete+compressive+strength>

The dataset has $N = 1030$ observations in total and the input features are $D = 8$ dimensional.

We will use a 90/10 split for training and testing.

Let's load and prepare the data using pandas:

Your job is now to repeat the analysis from part 1-3 for this data set.

Use $\alpha = \frac{1}{2}$, a step size of 10^{-2} , a batch size of 50, and 10000 training iterations for these experiments and the same network architecture as above.

Task 5.1: Fit a MAP-network to the data

Solution

```
Itt = 500/10000 (5.0%), log joint = -5521.36
Itt = 1000/10000 (10.0%), log joint = -5407.90
Itt = 1500/10000 (15.0%), log joint = -5315.85
Itt = 2000/10000 (20.0%), log joint = -5201.92
Itt = 2500/10000 (25.0%), log joint = -5044.16
Itt = 3000/10000 (30.0%), log joint = -4837.18
Itt = 3500/10000 (35.0%), log joint = -4649.94
Itt = 4000/10000 (40.0%), log joint = -4374.37
Itt = 4500/10000 (45.0%), log joint = -4089.93
Itt = 5000/10000 (50.0%), log joint = -3936.60
Itt = 5500/10000 (55.0%), log joint = -3866.58
Itt = 6000/10000 (60.0%), log joint = -3808.84
Itt = 6500/10000 (65.0%), log joint = -3758.70
Itt = 7000/10000 (70.0%), log joint = -3656.93
Itt = 7500/10000 (75.0%), log joint = -3579.62
Itt = 8000/10000 (80.0%), log joint = -3539.56
Itt = 8500/10000 (85.0%), log joint = -3528.18
Itt = 9000/10000 (90.0%), log joint = -3430.11
Itt = 9500/10000 (95.0%), log joint = -3383.74
Itt = 10000/10000 (100.0%), log joint = -3345.34
```

End of solution

Task 5.2: Compute the training and test average LPD using the MAP solution.

Solution

```
LPD Train: -2.61
LPD test: -5.40
```

End of solution

Task 5.3: Use the MAP solution to compute an LLLA approximation.

Solution

End of solution

Task 5.4: Compute the average training and test LPD for the LLLA solution.

Solution

LPD train LLLA: -2.61

LPD test LLLA: -3.94

End of solution

Task 5.5: Fit a deep ensemble with 10 models (it may take a little while to run, so start with a smaller number and increase the number of models when you are sure that everything is correct).

Solution

Training model 1/10

```
Itt = 500/10000 (5.0%), log joint = -5521.36
Itt = 1000/10000 (10.0%), log joint = -5407.90
Itt = 1500/10000 (15.0%), log joint = -5315.85
Itt = 2000/10000 (20.0%), log joint = -5201.92
Itt = 2500/10000 (25.0%), log joint = -5044.16
Itt = 3000/10000 (30.0%), log joint = -4837.18
Itt = 3500/10000 (35.0%), log joint = -4649.94
Itt = 4000/10000 (40.0%), log joint = -4374.37
Itt = 4500/10000 (45.0%), log joint = -4089.93
Itt = 5000/10000 (50.0%), log joint = -3936.60
Itt = 5500/10000 (55.0%), log joint = -3866.58
Itt = 6000/10000 (60.0%), log joint = -3808.84
Itt = 6500/10000 (65.0%), log joint = -3758.70
Itt = 7000/10000 (70.0%), log joint = -3656.93
Itt = 7500/10000 (75.0%), log joint = -3579.62
Itt = 8000/10000 (80.0%), log joint = -3539.56
Itt = 8500/10000 (85.0%), log joint = -3528.18
Itt = 9000/10000 (90.0%), log joint = -3430.11
Itt = 9500/10000 (95.0%), log joint = -3383.74
Itt = 10000/10000 (100.0%), log joint = -3345.34
```

Training model 2/10

```
Itt = 500/10000 (5.0%), log joint = -5527.48
Itt = 1000/10000 (10.0%), log joint = -5387.52
Itt = 1500/10000 (15.0%), log joint = -5280.08
Itt = 2000/10000 (20.0%), log joint = -5156.80
Itt = 2500/10000 (25.0%), log joint = -4985.83
Itt = 3000/10000 (30.0%), log joint = -4797.24
Itt = 3500/10000 (35.0%), log joint = -4515.87
Itt = 4000/10000 (40.0%), log joint = -4106.83
Itt = 4500/10000 (45.0%), log joint = -3920.02
Itt = 5000/10000 (50.0%), log joint = -3785.29
Itt = 5500/10000 (55.0%), log joint = -3701.31
Itt = 6000/10000 (60.0%), log joint = -3662.84
Itt = 6500/10000 (65.0%), log joint = -3565.64
```

```

Itt = 7000/10000 (70.0%), log joint = -3515.66
Itt = 7500/10000 (75.0%), log joint = -3482.74
Itt = 8000/10000 (80.0%), log joint = -3437.14
Itt = 8500/10000 (85.0%), log joint = -3402.75
Itt = 9000/10000 (90.0%), log joint = -3426.22
Itt = 9500/10000 (95.0%), log joint = -3299.79
Itt = 10000/10000 (100.0%), log joint = -3325.05
Training model 3/10
Itt = 500/10000 (5.0%), log joint = -12312.80
Itt = 1000/10000 (10.0%), log joint = -7376.15
Itt = 1500/10000 (15.0%), log joint = -6242.88
Itt = 2000/10000 (20.0%), log joint = -5845.74
Itt = 2500/10000 (25.0%), log joint = -5677.31
Itt = 3000/10000 (30.0%), log joint = -5599.85
Itt = 3500/10000 (35.0%), log joint = -5553.63
Itt = 4000/10000 (40.0%), log joint = -5521.71
Itt = 4500/10000 (45.0%), log joint = -5494.10
Itt = 5000/10000 (50.0%), log joint = -5470.80
Itt = 5500/10000 (55.0%), log joint = -5448.04
Itt = 6000/10000 (60.0%), log joint = -5425.28
Itt = 6500/10000 (65.0%), log joint = -5400.34
Itt = 7000/10000 (70.0%), log joint = -5373.07
Itt = 7500/10000 (75.0%), log joint = -5341.33
Itt = 8000/10000 (80.0%), log joint = -5303.99
Itt = 8500/10000 (85.0%), log joint = -5259.52
Itt = 9000/10000 (90.0%), log joint = -5204.57
Itt = 9500/10000 (95.0%), log joint = -5133.91
Itt = 10000/10000 (100.0%), log joint = -5038.83
Training model 4/10
Itt = 500/10000 (5.0%), log joint = -10295.54
Itt = 1000/10000 (10.0%), log joint = -6818.42
Itt = 1500/10000 (15.0%), log joint = -5975.09
Itt = 2000/10000 (20.0%), log joint = -5691.05
Itt = 2500/10000 (25.0%), log joint = -5584.99
Itt = 3000/10000 (30.0%), log joint = -5536.49
Itt = 3500/10000 (35.0%), log joint = -5509.06
Itt = 4000/10000 (40.0%), log joint = -5488.92
Itt = 4500/10000 (45.0%), log joint = -5470.01
Itt = 5000/10000 (50.0%), log joint = -5450.14
Itt = 5500/10000 (55.0%), log joint = -5428.34
Itt = 6000/10000 (60.0%), log joint = -5403.51
Itt = 6500/10000 (65.0%), log joint = -5374.31
Itt = 7000/10000 (70.0%), log joint = -5338.72
Itt = 7500/10000 (75.0%), log joint = -5294.53
Itt = 8000/10000 (80.0%), log joint = -5237.40
Itt = 8500/10000 (85.0%), log joint = -5158.73

```

Itt = 9000/10000 (90.0%), log joint = -5048.11
Itt = 9500/10000 (95.0%), log joint = -4904.49
Itt = 10000/10000 (100.0%), log joint = -4752.36

Training model 5/10

Itt = 500/10000 (5.0%), log joint = -5244.08
Itt = 1000/10000 (10.0%), log joint = -5043.35
Itt = 1500/10000 (15.0%), log joint = -4789.81
Itt = 2000/10000 (20.0%), log joint = -4400.44
Itt = 2500/10000 (25.0%), log joint = -4046.22
Itt = 3000/10000 (30.0%), log joint = -3938.17
Itt = 3500/10000 (35.0%), log joint = -3871.12
Itt = 4000/10000 (40.0%), log joint = -3711.22
Itt = 4500/10000 (45.0%), log joint = -3613.95
Itt = 5000/10000 (50.0%), log joint = -3560.07
Itt = 5500/10000 (55.0%), log joint = -3498.32
Itt = 6000/10000 (60.0%), log joint = -3457.54
Itt = 6500/10000 (65.0%), log joint = -3379.87
Itt = 7000/10000 (70.0%), log joint = -3436.26
Itt = 7500/10000 (75.0%), log joint = -3329.75
Itt = 8000/10000 (80.0%), log joint = -3376.54
Itt = 8500/10000 (85.0%), log joint = -3193.94
Itt = 9000/10000 (90.0%), log joint = -3154.67
Itt = 9500/10000 (95.0%), log joint = -3173.69
Itt = 10000/10000 (100.0%), log joint = -3072.59

Training model 6/10

Itt = 500/10000 (5.0%), log joint = -5881.11
Itt = 1000/10000 (10.0%), log joint = -5558.38
Itt = 1500/10000 (15.0%), log joint = -5461.06
Itt = 2000/10000 (20.0%), log joint = -5401.34
Itt = 2500/10000 (25.0%), log joint = -5352.07
Itt = 3000/10000 (30.0%), log joint = -5307.20
Itt = 3500/10000 (35.0%), log joint = -5258.67
Itt = 4000/10000 (40.0%), log joint = -5202.75
Itt = 4500/10000 (45.0%), log joint = -5138.82
Itt = 5000/10000 (50.0%), log joint = -5068.41
Itt = 5500/10000 (55.0%), log joint = -4999.54
Itt = 6000/10000 (60.0%), log joint = -4939.02
Itt = 6500/10000 (65.0%), log joint = -4882.47
Itt = 7000/10000 (70.0%), log joint = -4805.85
Itt = 7500/10000 (75.0%), log joint = -4707.26
Itt = 8000/10000 (80.0%), log joint = -4538.53
Itt = 8500/10000 (85.0%), log joint = -4231.75
Itt = 9000/10000 (90.0%), log joint = -3978.84
Itt = 9500/10000 (95.0%), log joint = -3798.01
Itt = 10000/10000 (100.0%), log joint = -3711.03

Training model 7/10

```

Itt = 500/10000 (5.0%), log joint = -17572.23
Itt = 1000/10000 (10.0%), log joint = -8674.62
Itt = 1500/10000 (15.0%), log joint = -6683.92
Itt = 2000/10000 (20.0%), log joint = -5977.65
Itt = 2500/10000 (25.0%), log joint = -5674.67
Itt = 3000/10000 (30.0%), log joint = -5538.88
Itt = 3500/10000 (35.0%), log joint = -5473.31
Itt = 4000/10000 (40.0%), log joint = -5437.59
Itt = 4500/10000 (45.0%), log joint = -5412.67
Itt = 5000/10000 (50.0%), log joint = -5390.15
Itt = 5500/10000 (55.0%), log joint = -5365.76
Itt = 6000/10000 (60.0%), log joint = -5336.04
Itt = 6500/10000 (65.0%), log joint = -5299.65
Itt = 7000/10000 (70.0%), log joint = -5254.43
Itt = 7500/10000 (75.0%), log joint = -5198.20
Itt = 8000/10000 (80.0%), log joint = -5126.54
Itt = 8500/10000 (85.0%), log joint = -5041.30
Itt = 9000/10000 (90.0%), log joint = -4948.86
Itt = 9500/10000 (95.0%), log joint = -4858.21
Itt = 10000/10000 (100.0%), log joint = -4780.05
Training model 8/10
Itt = 500/10000 (5.0%), log joint = -5317.97
Itt = 1000/10000 (10.0%), log joint = -5209.30
Itt = 1500/10000 (15.0%), log joint = -5071.88
Itt = 2000/10000 (20.0%), log joint = -4891.83
Itt = 2500/10000 (25.0%), log joint = -4698.33
Itt = 3000/10000 (30.0%), log joint = -4392.52
Itt = 3500/10000 (35.0%), log joint = -4078.65
Itt = 4000/10000 (40.0%), log joint = -3956.58
Itt = 4500/10000 (45.0%), log joint = -3898.45
Itt = 5000/10000 (50.0%), log joint = -3860.84
Itt = 5500/10000 (55.0%), log joint = -3776.89
Itt = 6000/10000 (60.0%), log joint = -3709.25
Itt = 6500/10000 (65.0%), log joint = -3675.32
Itt = 7000/10000 (70.0%), log joint = -3521.25
Itt = 7500/10000 (75.0%), log joint = -3466.30
Itt = 8000/10000 (80.0%), log joint = -3523.86
Itt = 8500/10000 (85.0%), log joint = -3371.66
Itt = 9000/10000 (90.0%), log joint = -3409.93
Itt = 9500/10000 (95.0%), log joint = -3312.74
Itt = 10000/10000 (100.0%), log joint = -3264.82
Training model 9/10
Itt = 500/10000 (5.0%), log joint = -6814.67
Itt = 1000/10000 (10.0%), log joint = -5713.79
Itt = 1500/10000 (15.0%), log joint = -5553.66
Itt = 2000/10000 (20.0%), log joint = -5511.13

```

```

Itt = 2500/10000 (25.0%), log joint = -5482.39
Itt = 3000/10000 (30.0%), log joint = -5450.16
Itt = 3500/10000 (35.0%), log joint = -5409.20
Itt = 4000/10000 (40.0%), log joint = -5355.25
Itt = 4500/10000 (45.0%), log joint = -5283.72
Itt = 5000/10000 (50.0%), log joint = -5178.01
Itt = 5500/10000 (55.0%), log joint = -5011.46
Itt = 6000/10000 (60.0%), log joint = -4802.61
Itt = 6500/10000 (65.0%), log joint = -4664.53
Itt = 7000/10000 (70.0%), log joint = -4401.16
Itt = 7500/10000 (75.0%), log joint = -4084.23
Itt = 8000/10000 (80.0%), log joint = -3994.36
Itt = 8500/10000 (85.0%), log joint = -3909.82
Itt = 9000/10000 (90.0%), log joint = -3794.03
Itt = 9500/10000 (95.0%), log joint = -3717.29
Itt = 10000/10000 (100.0%), log joint = -3637.35
Training model 10/10
Itt = 500/10000 (5.0%), log joint = -7814.67
Itt = 1000/10000 (10.0%), log joint = -5958.74
Itt = 1500/10000 (15.0%), log joint = -5589.82
Itt = 2000/10000 (20.0%), log joint = -5481.85
Itt = 2500/10000 (25.0%), log joint = -5437.73
Itt = 3000/10000 (30.0%), log joint = -5410.75
Itt = 3500/10000 (35.0%), log joint = -5387.16
Itt = 4000/10000 (40.0%), log joint = -5361.76
Itt = 4500/10000 (45.0%), log joint = -5333.44
Itt = 5000/10000 (50.0%), log joint = -5300.36
Itt = 5500/10000 (55.0%), log joint = -5259.99
Itt = 6000/10000 (60.0%), log joint = -5208.34
Itt = 6500/10000 (65.0%), log joint = -5138.87
Itt = 7000/10000 (70.0%), log joint = -5044.82
Itt = 7500/10000 (75.0%), log joint = -4919.47
Itt = 8000/10000 (80.0%), log joint = -4795.95
Itt = 8500/10000 (85.0%), log joint = -4625.79
Itt = 9000/10000 (90.0%), log joint = -4341.65
Itt = 9500/10000 (95.0%), log joint = -4026.49
Itt = 10000/10000 (100.0%), log joint = -3883.91

```

End of solution

Task 5.6: Evaluate the training and test LPDs for the DE.

Solution

LPD train DE: -2.82

LPD test DE: -3.26

End of solution

Task 5.7: Which of the three methods (MAP, LLLA, DE) works best for this data set? [Discussion question]

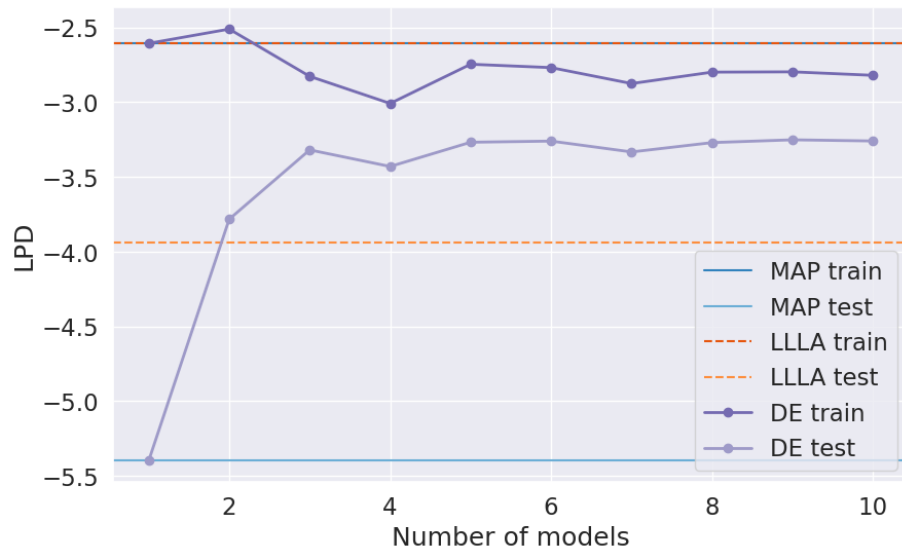
Task 5.8: Plot the performance as a function of the number of models in the ensemble. Plot the performance of the MAP solution as a horizontal line for reference. How well does the DE compare in terms of training and test performance?

Hints

- First compute predictions for an ensemble with $S = 1$ model (i.e. the MAP solution), then $S = 2$, $S = 3$ etc.
- You only need to fit your ensemble once. After you've fitted the ensemble you can make predictions using subsets of the models using the `subset_models` argument to the `predict` function.

Solution

<matplotlib.legend.Legend at 0x7f6d83993470>



End of solution